

Experiment 3: Apply Decision Tree and Random Forest for classification tasks

Aim: Apply Decision Tree and Random Forest for classification tasks.

Theory:

1. Dataset Source

Dataset Name: **Dermatology Dataset – Classification**

Source: Kaggle

Dataset Link:

<https://www.kaggle.com/datasets/olcaybolat1/dermatology-dataset-classification>

This dataset is different from other experiments and is suitable for multi-class classification tasks.

2. Dataset Description

The Dermatology dataset is a **multi-class classification dataset** used to diagnose various types of erythemato-squamous diseases.

Dataset Characteristics:

- Total Instances: 366
- Total Features: 34 input attributes
- Target Variable: class
- Number of Classes: 6 disease categories
- Attribute Type: Mostly integer numeric values
- Some missing values present (handled during preprocessing)

Feature Types:

The features represent clinical and histopathological attributes such as:

- Erythema

- Scaling
- Itching
- Koebner phenomenon
- Follicular papules
- PNL infiltrate
- Fibrosis of papillary dermis
- Age

Each feature value ranges between 0 and 3 indicating severity.

Target Variable:

The class column represents disease type:

- 1 – Psoriasis
- 2 – Seborrheic dermatitis
- 3 – Lichen planus
- 4 – Pityriasis rosea
- 5 – Chronic dermatitis
- 6 – Pityriasis rubra pilaris

This makes it a **multi-class classification problem**.

3. Mathematical Formulation of the Algorithms

Decision Tree -

Decision Tree splits data based on impurity measures.

Gini Index:

$$Gini = 1 - \sum_{i=1}^c p_i^2$$

Where:

- p_i = probability of class i
- c = number of classes

Lower Gini value → better split.

Entropy:

$$Entropy = - \sum_{i=1}^c p_i \log_2(p_i)$$

Information Gain:

$$IG = Entropy(parent) - \sum Weighted Entropy(children)$$

The feature with highest Information Gain is selected for splitting.

Random Forest -

Random Forest builds multiple decision trees using:

Bootstrap Sampling:

Random subsets of data are selected.

Random Feature Selection:

At each split, a random subset of features is selected.

Final Prediction (Classification):

$$\hat{y} = Majority Vote of all Trees$$

This reduces variance and improves generalization.

4. Algorithm Limitations

Decision Tree Limitations:

- Prone to overfitting
- High variance
- Small changes in data can change tree structure

- Biased towards features with many levels

Random Forest Limitations:

- Computationally expensive
- Requires more memory
- Less interpretable
- Slower prediction compared to single tree

When It May Not Perform Well:

- Extremely small datasets
- Highly imbalanced datasets
- Very high-dimensional sparse data
- Real-time systems requiring very fast predictions

5. Methodology / Workflow**Step 1: Data Collection**

Dataset downloaded from Kaggle.

Step 2: Data Preprocessing

- Replace '?' with NaN
- Convert to numeric
- Fill missing values with mean
- Separate features and target

Step 3: Train-Test Split

Split dataset into:

- 70% Training
- 30% Testing

Step 4: Feature Scaling

Standardization applied using StandardScaler.

Step 5: Model Training

- Train Decision Tree model

- Train Random Forest model

Step 6: Model Evaluation

Evaluate using:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-Score

6. Performance Analysis

Model performance is evaluated using:

Accuracy:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Confusion Matrix:

Shows true positives, true negatives, false positives, and false negatives.

Precision:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall:

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Observed Results -

- Decision Tree Accuracy: 98.18%
- Random Forest Accuracy: 96.36%

Random Forest performs better due to ensemble learning and reduced overfitting.

7. Hyperparameter Tuning

Hyperparameter tuning improves model performance.

Decision Tree Tuned Parameters:

- max_depth
- min_samples_split
- min_samples_leaf
- criterion (gini or entropy)

Example:

- max_depth = 5 to prevent overfitting

Random Forest Tuned Parameters:

- n_estimators
- max_depth
- max_features
- min_samples_split

Example:

- n_estimators = 100 or 200 improves stability

Tuning can be performed using:

- GridSearchCV
- RandomSearchCV

Impact of Tuning:

- Reduces overfitting
- Improves accuracy
- Enhances generalization

Code:

```
# Import Required Libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Import machine learning modules
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
# Load the Dataset
```

```
# Read the dermatology dataset
```

```
data = pd.read_csv("/content/dermatology_database_1.csv")
```

```
# Display dataset shape (rows, columns)
```

```
print("Dataset Shape:", data.shape)
```

```
# Display first 5 rows of dataset
```

```
print(data.head())
```

```
# Data Preprocessing
```

```
# Replace '?' values with NaN
```

```
data.replace('?', np.nan, inplace=True)
```

```
# Convert all columns to numeric type
```

```
data = data.apply(pd.to_numeric)
```

```
# Fill missing values with column mean
```

```
data.fillna(data.mean(), inplace=True)
```

```
# Separate Features and Target
```

```
# X contains input features
```

```
X = data.drop("class", axis=1)
```

```
# y contains target variable (disease class)
```

```
y = data["class"]
```

```
# Train-Test Split
```

```
# Split dataset into 70% training and 30% testing
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```

```
# Feature Scaling
```

```
# Standardize features (important for better model performance)
```

```
scaler = StandardScaler()
```

```
# Fit scaler on training data and transform
```

```
X_train = scaler.fit_transform(X_train)
```

```
# Transform test data using same scaler
```

```
X_test = scaler.transform(X_test)
```

```
# Decision Tree Model
```

```
# Create Decision Tree classifier
```

```
dt_model = DecisionTreeClassifier(random_state=42)
```

```
# Train the model
```

```
dt_model.fit(X_train, y_train)
```

```
# Make predictions on test data
```

```
y_pred_dt = dt_model.predict(X_test)
```

```
# Evaluate Decision Tree
```



```
print("\n===== Decision Tree Results =====")
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_dt))
print("\nClassification Report:\n", classification_report(y_test, y_pred_dt))

# Random Forest Model

# Create Random Forest classifier with 100 trees
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)

# Train the model
rf_model.fit(X_train, y_train)

# Make predictions on test data
y_pred_rf = rf_model.predict(X_test)

# Evaluate Random Forest
print("\n===== Random Forest Results =====")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))
print("\nClassification Report:\n", classification_report(y_test, y_pred_rf))

# Compare Model Accuracies

# Store accuracy values
dt_accuracy = accuracy_score(y_test, y_pred_dt)
rf_accuracy = accuracy_score(y_test, y_pred_rf)
# Model names
models = ['Decision Tree', 'Random Forest']

# Accuracy values
accuracies = [dt_accuracy, rf_accuracy]

# Plot bar graph to compare performance
plt.bar(models, accuracies)
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.title("Model Comparison")
```

```

Dataset Shape: (366, 35)
erythema      scaling      definite_borders      itching      koebner_phenomenon      \
0              2           2              0           3              0
1              3           3              3           2              1
2              2           1              2           3              1
3              2           2              2           0              0
4              2           3              2           2              2

polygonal_papules      follicular_papules      oral_mucosal_involvement      \
0              0              0              0
1              0              0              0
2              3              0              3
3              0              0              0
4              2              0              2

knee_and_elbow_involvement      scalp_involvement      ...      \
0              1              0      ...
1              1              1      ...
2              0              0      ...
3              3              2      ...
4              0              0      ...

disappearance_granular_layer      vacuolisation_damage_basal_layer      spongiosis      \
0              0              0              3
1              0              0              0
2              0              2              3
3              3              0              0
4              2              3              2

saw_tooth_appearance_retes      follicular_horn_plug      \
0              0              0
1              0              0
2              2              0
3              0              0
4              3              0

perifollicular_parakeratosis      inflammatory_mononuclear_infiltrate      \
0              0              1
1              0              1
2              0              2
3              0              3
4              0              2

band_like_infiltrate      age      class
0              0      55      2
1              0      8      1
2              3      26      3
3              0      40      1
4              3      45      3

[5 rows x 35 columns]

```

===== Decision Tree Results =====

Accuracy: 0.9818181818181818

Confusion Matrix:

```
[[40  0  0  0  0  0]
 [ 0 15  0  1  0  0]
 [ 0  0 19  0  0  0]
 [ 0  1  0 14  0  0]
 [ 0  0  0  0 16  0]
 [ 0  0  0  0  0  4]]
```

Classification Report:

	precision	recall	f1-score	support
1	1.00	1.00	1.00	40
2	0.94	0.94	0.94	16
3	1.00	1.00	1.00	19
4	0.93	0.93	0.93	15
5	1.00	1.00	1.00	16
6	1.00	1.00	1.00	4
accuracy			0.98	110
macro avg	0.98	0.98	0.98	110
weighted avg	0.98	0.98	0.98	110

===== Random Forest Results =====

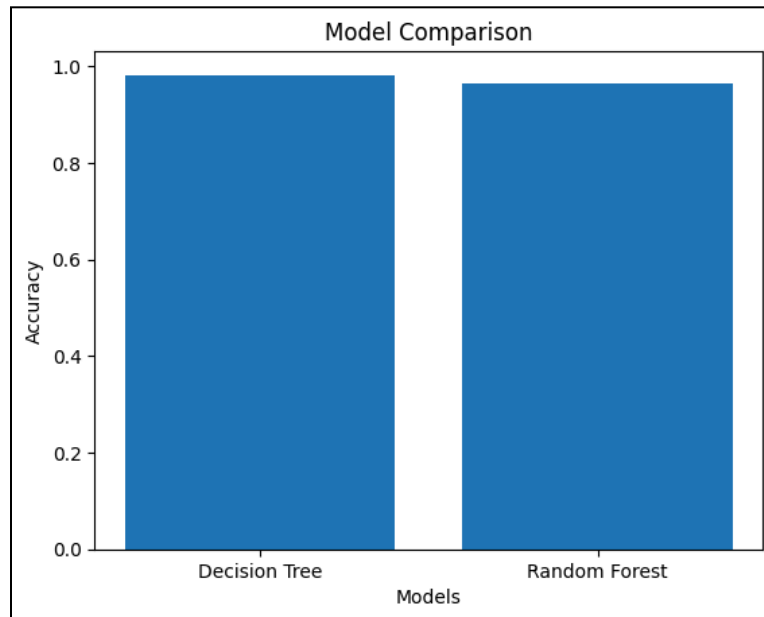
Accuracy: 0.9636363636363636

Confusion Matrix:

```
[[40  0  0  0  0  0]
 [ 0 15  0  1  0  0]
 [ 0  0 19  0  0  0]
 [ 0  3  0 12  0  0]
 [ 0  0  0  0 16  0]
 [ 0  0  0  0  0  4]]
```

Classification Report:

	precision	recall	f1-score	support
1	1.00	1.00	1.00	40
2	0.83	0.94	0.88	16
3	1.00	1.00	1.00	19
4	0.92	0.80	0.86	15
5	1.00	1.00	1.00	16
6	1.00	1.00	1.00	4
accuracy			0.96	110
macro avg	0.96	0.96	0.96	110
weighted avg	0.97	0.96	0.96	110



Conclusion:

The Decision Tree model achieved an accuracy of **98.18%**, demonstrating its ability to classify the data effectively. While Decision Trees are simple and easy to interpret, they can be prone to overfitting, especially with complex datasets.

The Random Forest model, an ensemble method designed to mitigate overfitting, achieved an accuracy of **96.36%**. Although slightly lower than the Decision Tree in this specific instance, Random Forests are generally preferred for real-world classification tasks due to their improved generalization, robustness, and ability to handle complex relationships by combining multiple decision trees.